



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

[Hardening de linux]

P R E S E N T A:

[Oscar Alberto Leal Ramírez][22590042]

[Isaac Castro Islas][B19140346]

[Hortencia Sanchez Carlos][B23590532]

[Ingeniería en Sistemas Computacionales]

Equipo Lima

PERIODO [Enero-Junio (2026)]



REPORTE DE PRÁCTICA - HARDENING DE GNU/LINUX

Aseguramiento de acceso remoto, control de privilegios, firewall, autenticación multifactor y respaldos automatizados

Objetivo:

El objetivo de esta práctica fue implementar un proceso de hardening sobre un servidor GNU/Linux, fortaleciendo el acceso remoto por SSH, la autenticación mediante llaves criptográficas, la restricción del usuario root, la aplicación de reglas de firewall, la administración de usuarios nominales, la autenticación de dos factores y la continuidad operativa mediante respaldos automatizados bajo la regla 3-2-1. La práctica se documentó con evidencia visual de cada fase para justificar técnicamente los cambios realizados y demostrar que cada control fue aplicado de forma ordenada, verificable y alineada con el principio de menor privilegio.

Infraestructura utilizada:

Componente	Descripción
Servidor principal	Debian 12 GNU/Linux utilizado como servidor endurecido y nodo principal de administración.
Cliente de auditoría	Kali Linux empleado para pruebas de conexión SSH, validaciones y revisión de servicios.
Sistema operativo base	GNU/Linux Debian con administración mediante usuario nominal y privilegios controlados con sudo.
Acceso remoto	OpenSSH configurado en el puerto 12222, con autenticación por llaves criptográficas y bloqueo de acceso directo como root.
Firewall perimetral	UFW configurado con política restrictiva: denegar conexiones entrantes por defecto y permitir únicamente servicios necesarios.
Autenticación adicional	Google Authenticator / TOTP integrado con PAM para reforzar el inicio de sesión SSH.
Servidor de respaldo	Windows Server 2022 conectado mediante Tailscale y OpenSSH para recibir copias remotas en C:\RespaldosLinux.
Almacenamiento externo	Google Drive configurado con rclone como respaldo fuera de sitio dentro de gdrive:RespaldosLinux.
Automatización	Cron ejecutando /opt/backup_321.sh todos los días a la 1:01 AM, con registro en /var/log/backup_cron.log.
Red privada	Tailscale como red privada entre Debian y Windows Server para transferencias internas sin exposición directa a Internet.

La infraestructura utilizada permitió simular un entorno real de administración segura, comunicación privada entre nodos y respaldo distribuido bajo la regla 3-2-1.

Desarrollo de la práctica

A continuación, se presentan los pasos realizados durante el endurecimiento del servidor, manteniendo la secuencia operativa original y las evidencias correspondientes a cada configuración aplicada.

Paso 1: Conexión remota por SSH e intercambio de llaves:

Comando ejecutado:

```
ssh -p 12222 admnytc@159.223.152.22
```

```
(oscar@kali)-[~]
$ ssh -p 12222 admnyc@159.223.152.22
The authenticity of host '[159.223.152.22]:12222 ([159.223.152.22]:12222)' can't be established.
ED25519 key fingerprint is: SHA256:fpF6VAv4zLMLjyM0GEuKUXWk2Rmd261Z+bIwpKiHdWI
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[159.223.152.22]:12222' (ED25519) to the list of known hosts.
admnyc@159.223.152.22's password:
Linux debian-ici 6.12.43+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.43-1 (2025-08-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

Figura 1. Proceso de autenticación e inicio de sesión SSH.

Se realizó la conexión por SSH desde Kali hacia el servidor Debian utilizando el puerto 12222. Posteriormente, se aceptó la huella digital del servidor y se inició sesión con el usuario admnyc en lugar de root. Estas acciones se implementaron para mejorar la seguridad, ya que el cambio de puerto reduce ataques automáticos al puerto 22, la verificación del fingerprint protege contra ataques Man-in-the-Middle y el uso de un usuario estándar evita el acceso directo como administrador.

¿Por qué usamos Máquinas Virtuales (VM)?

Se utilizaron máquinas virtuales para crear un entorno de pruebas seguro y controlado. Kali Linux permitió realizar auditorías y pruebas sin afectar el sistema operativo principal, ya que cualquier error o cambio quedó aislado dentro de la VM. Además, este entorno facilitó la simulación de un escenario real de red entre cliente y servidor sin necesidad de utilizar hardware físico adicional.

Paso 2: Intento de escaneo de puertos (Reconocimiento):

Comando:

sudo nmps -sV -p- -T4 159.223.152.22 (Nota: El comando correcto es nmap)

```
admnyc@debian-ici:~$ sudo nmap -sV -p- -T4 159.223.152.22
[sudo] contraseña para admnyc:
sudo: nmap: command not found
admnyc@debian-ici:~$ sudo nmap -sV -p- -T4 159.223.152.22
sudo: nmap: command not found
```

Figura 2. Error de comando no encontrado al ejecutar Nmap.

Se intentó ejecutar un escaneo de puertos con nmap desde el servidor Debian hacia su propia IP pública, pero el sistema mostró el error command not found porque la herramienta no estaba instalada.

Explicación:

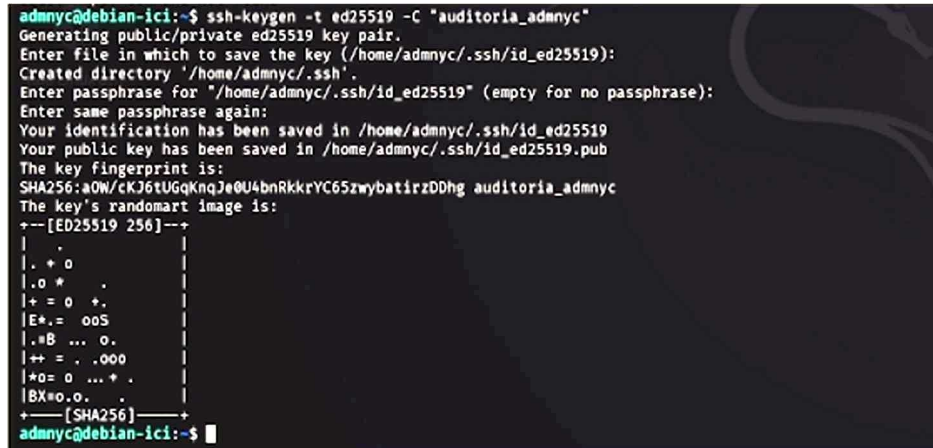
El escaneo debía realizarse desde una máquina externa, como Kali Linux, y no desde el mismo servidor. Además, que nmap no esté instalado es una buena práctica de seguridad, ya que reduce herramientas

que podrían ser usadas por un atacante en caso de comprometer el sistema.

Paso 3: Generación de par de llaves SSH (Autenticación Robusta):

Comando:

```
ssh-keygen -t ed25519 -C "auditoria_admnc"
```



```
admnc@debian-ici:~$ ssh-keygen -t ed25519 -C "auditoria_admnc"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/admnc/.ssh/id_ed25519):
Created directory '/home/admnc/.ssh'.
Enter passphrase for '/home/admnc/.ssh/id_ed25519' (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/admnc/.ssh/id_ed25519
Your public key has been saved in /home/admnc/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:aOW/cKJ6tUGqKnQJe0U4bnRkkrYC65zwybatirzDDhg auditoria_admnc
The key's randomart image is:
+--[ED25519 256]--+
|
|..+ 0
|.o *
|+ = 0 +.
|E+.= ooS
|.B ... o.
|++ = . .000
|*o= 0 ... +
|BX=o.o.
+---[SHA256]-----
admnc@debian-ici:~$
```

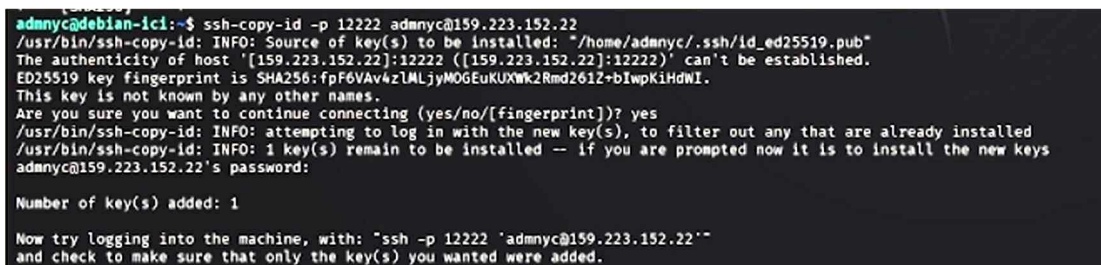
Figura 3. Creación de llaves SSH para autenticación robusta.

Se generó un par de llaves criptográficas para el usuario admnc: una llave privada almacenada localmente y una llave pública destinada al servidor. Se utilizaron las opciones predeterminadas y no se configuró contraseña adicional para facilitar la automatización. Se utilizó el algoritmo ED25519 por ser uno de los métodos de cifrado más seguros y eficientes actualmente. Además, el uso de llaves criptográficas permite desactivar el acceso mediante contraseñas tradicionales, fortaleciendo la seguridad del servidor y reduciendo el riesgo de ataques por fuerza bruta.

Paso 4: Copia e instalación de la llave pública en el servidor

Comando:

```
ssh-copy-id -p 12222 admnc@159.223.152.22
```



```
admnc@debian-ici:~$ ssh-copy-id -p 12222 admnc@159.223.152.22
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/admnc/.ssh/id_ed25519.pub"
The authenticity of host '[159.223.152.22]:12222 ([159.223.152.22]:12222)' can't be established.
ED25519 key fingerprint is SHA256:fpF6VAv4zLALjyMOGEuKUXWk2Rmd261Z+bIwpKiHdWI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
admnc@159.223.152.22's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -p 12222 'admnc@159.223.152.22'"
and check to make sure that only the key(s) you wanted were added.
```

Figura 4. Almacenamiento de llave SSH en el host remoto.

Se instaló automáticamente la llave pública en el servidor remoto mediante ssh-copy-id usando el puerto

12222. El sistema confirmó la conexión y agregó correctamente la llave autorizada. Se utilizó ssh-copy-id porque permite copiar la llave pública de forma segura al archivo `authorized_keys` del servidor. Con esto, el acceso puede realizarse mediante autenticación por llaves criptográficas, evitando el uso de contraseñas y mejorando la seguridad del sistema.

Paso 5: Modificación de la configuración del servidor SSH:

Comando:

```
sudo nano /etc/ssh/sshd_config
```



```
adnyc@debian-ici:~$ sudo nano /etc/ssh/sshd_config
```

Figura 5. Acceso a los parámetros de configuración de SSH.

Se abrió el archivo principal de configuración del servicio SSH (`sshd_config`) utilizando el editor Nano con permisos de administrador. Este archivo permite modificar parámetros de seguridad del acceso remoto, como cambiar el puerto por defecto y deshabilitar el acceso directo del usuario root. Se utilizó `sudo` porque el archivo se encuentra en una ruta protegida del sistema y requiere privilegios elevados para realizar cambios.

Paso 6: Mitigación de ataques de fuerza bruta por cambio de puerto por defecto:

Archivo modificado:

```
/etc/ssh/sshd_config (Línea: Port 12222)
```



```
Include /etc/ssh/sshd_config.d/*.conf

Port 12222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key
```

Se modificó el archivo de configuración de SSH para cambiar el puerto predeterminado 22 por el puerto personalizado 12222, y posteriormente se guardaron los cambios. El cambio de puerto ayuda a reducir intentos automáticos de ataque dirigidos al puerto 22, que es el más utilizado por defecto en SSH. Con esto, se mejora la seguridad del servidor al dificultar accesos no autorizados y obligar a especificar el nuevo puerto para conectarse.

Paso 7: Prohibir explícitamente el login del superusuario (Mitigación de escalación de privilegios):

Archivo modificado:

/etc/ssh/sshd_config (Línea: PermitRootLogin no)

```
# Authentication:
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes
```

Figura 7. Restricción de acceso SSH al usuario administrador root.

Se modificó la directiva PermitRootLogin en la configuración de SSH, activándola y estableciendo su valor en no para deshabilitar el acceso remoto directo del usuario root. Esta configuración evita que el superusuario pueda iniciar sesión directamente desde SSH, reduciendo riesgos de ataques contra la cuenta root. Ahora, la administración del servidor debe realizarse mediante un usuario normal con permisos sudo, permitiendo mayor control y registro de actividades.

Paso 8: Restricción absoluta de autenticación por contraseña tradicional:

Archivo modificado:

/etc/ssh/sshd_config (Líneas: PasswordAuthentication no y PermitEmptyPasswords no)

```
# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication no
PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication. Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
# Beware issues with some PAM modules and threads.
```

Figura 8. Restricción de acceso SSH mediante contraseñas.

Se modificaron las directivas de autenticación en la configuración de SSH para deshabilitar el acceso mediante contraseñas y bloquear cuentas sin contraseña. Con estas configuraciones, el servidor solo permite autenticación mediante llaves criptográficas, aumentando la seguridad del acceso remoto. Además, se evita el uso de contraseñas vacías y se bloquean ataques de fuerza bruta basados en el intento masivo de contraseñas.

Paso 9: Forzar autenticación criptográfica por llaves públicas:

Archivo modificado

/etc/ssh/sshd_config (Línea: PubkeyAuthentication yes)


```
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2
```

Figura 9. Habilitación explícita de la autenticación por llave pública.

Se activó la directiva PubkeyAuthentication en la configuración de SSH y se estableció su valor en yes para permitir la autenticación mediante llaves criptográficas. Esta configuración habilita el acceso seguro usando las llaves previamente generadas. De esta manera, el servidor verifica automáticamente la identidad del usuario mediante el archivo authorized_keys, evitando el uso de contraseñas y reforzando la seguridad del acceso remoto.

Paso 10: Control de persistencia y desconexión por inactividad (Defensa en profundidad):

Archivo modificado:

/etc/ssh/sshd_config (Líneas: ClientAliveInterval 300 y ClientAliveCountMax 2)

```
# ForceCommand cvs server
ClientAliveInterval 300
ClientAliveCountMax 2
```

Figura 10. Configuración de tiempos de desconexión por inactividad.

Se configuraron parámetros de tiempo de espera en SSH para cerrar automáticamente sesiones inactivas después de cierto periodo. Estas reglas permiten que el servidor detecte sesiones sin actividad y las cierre automáticamente tras varios minutos de inactividad. Con ello, se reduce el riesgo de accesos no autorizados en equipos que hayan quedado abiertos o desatendidos.

Se guardaron los cambios realizados en el archivo de configuración utilizando Ctrl + O, se confirmó con Enter y posteriormente se cerró el editor Nano con Ctrl + X.

Paso 11: Aplicación de cambios en SSH y endurecimiento perimetral mediante Firewall (UFW):

Comandos ejecutados:

```
sudo sshd -t
sudo systemctl restart ssh sudo ufw default deny incoming
sudo ufw default allow outgoing sudo ufw allow 1222/tcp comment 'Acceso SSH Administrativo'
sudo ufw enable
```

```

admnyca@debian-ici:~$ sudo nano /etc/ssh/sshd_config
admnyca@debian-ici:~$ sudo sshd -t && sudo systemctl restart ssh
admnyca@debian-ici:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
admnyca@debian-ici:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
admnyca@debian-ici:~$ sudo ufw allow 12222/tcp comment 'Acceso SSH Administrativo'
Rule added
Rule added (v6)
admnyca@debian-ici:~$ sudo ufw enable
Firewall is active and enabled on system startup

```

Figura 11. Reinicio de SSH y endurecimiento perimetral con UFW.

Se verificó la configuración del servicio SSH para comprobar que no existieran errores antes de reiniciarlo. Después, se configuró el firewall ufw estableciendo reglas de seguridad restrictivas: bloquear todas las conexiones entrantes por defecto, permitir conexiones salientes y habilitar únicamente el acceso al puerto 12222. Finalmente, se activó el firewall para que las reglas quedaran funcionando de manera permanente.

- Primero se validó el archivo de configuración de SSH para evitar errores que pudieran impedir futuras conexiones al servidor. Una vez comprobado, el servicio fue reiniciado para aplicar los cambios realizados.
- Posteriormente, se configuró el firewall siguiendo una política de “todo bloqueado excepto lo necesario”. Esto permite reducir la superficie de ataque, ya que cualquier conexión no autorizada será rechazada automáticamente antes de llegar al sistema.
- También se permitió el tráfico saliente para que el servidor pueda seguir descargando actualizaciones y paquetes de seguridad. Además, únicamente se abrió el puerto 12222, utilizado para la administración remota mediante SSH.
- Finalmente, al habilitar ufw, el firewall quedó activo y configurado para iniciarse automáticamente cada vez que el servidor se reinicie, garantizando la permanencia de las medidas de seguridad aplicadas.

Paso 12: Creación de usuarios nominales independientes para administración:

Comando:

```
sudo adduser oscar_admin
```



```

admny@debian-ici:~$ sudo adduser oscar_admin
Nueva contraseña:
Vuelva a escribir la nueva contraseña:
passwd: contraseña actualizada correctamente
Cambiando la información de usuario para oscar_admin
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo []: oscar leal
Número de habitación []:
Teléfono del trabajo []:
Teléfono de casa []:
Otro []:
Is the information correct? [Y/n] n
Cambiando la información de usuario para oscar_admin
Introduzca el nuevo valor, o pulse INTRO para usar el valor predeterminado
Nombre completo [oscar leal]: oscar leal
Número de habitación []: 1
Teléfono del trabajo []: 4272010217
Teléfono de casa []: 4272010217
Otro []:
Is the information correct? [Y/n] Y

```

Figura 12. Creación de un usuario administrador nominal en el sistema.

Se creó una nueva cuenta de usuario llamada oscar_admin utilizando el comando adduser con privilegios de administrador. Durante el proceso se asignó una contraseña y se registraron los datos básicos del usuario.

La creación de cuentas individuales permite mantener un mejor control y registro de las actividades realizadas en el servidor.

De esta forma, cada acción queda asociada a un usuario específico dentro de los archivos de eventos del sistema. Además, separar las cuentas administrativas mejora la seguridad y la administración de accesos, ya que permite habilitar o deshabilitar usuarios de manera individual sin afectar al resto del equipo.

Paso 13: Intento de asignación de privilegios y auditoría de comandos mediante ayuda nativa (Error):

Comando ejecutado:

```
sudo usermod -a6 sudo oscar_admin
```

```
admny@debian-ici:~$ sudo usermod -a6 sudo oscar_admin
usermod: opción inválida -- '6'
Modo de uso: usermod [opciones] USUARIO

Opciones:
-a, --append                append the user to the supplemental GROUPS
                             mentioned by the -G option without removing
                             the user from other groups
-b, --badname               allow bad names (DEPRECATED)
-c, --comment COMENTARIO   nuevo valor del campo GECOS
-d, --home DIR_PERSONAL    nuevo directorio personal del nuevo usuario
-e, --expiredate FECHA_EXPIR establece la fecha de caducidad de la
                             cuenta a FECHA_EXPIR
-f, --inactive INACTIVO    establece el tiempo de inactividad después
                             de que caduque la cuenta a INACTIVO
-g, --gid GRUPO            fuerza el uso de GRUPO para la nueva cuenta
                             de usuario
-G, --groups GRUPOS        lista de grupos suplementarios
-h, --help                 muestra este mensaje de ayuda y termina
-l, --login NOMBRE         nuevo nombre para el usuario
-L, --lock                 bloquea la cuenta de usuario
-m, --move-home            mueve los contenidos del directorio
                             personal al directorio nuevo (usar sólo
                             junto con -d)
-o, --non-unique            permite usar UID duplicados (no únicos)
-p, --password CONTRASEÑA  usar la contraseña cifrada para la nueva cuenta
-P, --prefix PREFIX_DIR    prefix directory where are located the /etc/* files
-r, --remove               remove the user from only the supplemental GROUPS
                             mentioned by the -G option without removing
                             the user from other groups
-R, --root DIR_CHROOT      establece DIR_CHROOT como el directorio
                             al cual hacer chroot
-s, --shell CONSOLA        nueva consola de acceso para la cuenta del
                             usuario
-u, --uid UID              fuerza el uso del UID para la nueva cuenta
                             de usuario
-U, --unlock               desbloquea la cuenta de usuario
-v, --add-subuids FIRST-LAST add range of subordinate uids
-V, --del-subuids FIRST-LAST remove range of subordinate uids
-w, --add-subgids FIRST-LAST add range of subordinate gids
-W, --del-subgids FIRST-LAST remove range of subordinate gids
-Z, --selinux-user SEUSER  new SELinux user mapping for the user account
    --selinux-range SERANGE new SELinux MLS range for the user account

admny@debian-ici:~$
```

Figura 13. Error de sintaxis y despliegue de ayuda en usermod.

Se intentó agregar al usuario oscar_admin al grupo sudo para otorgarle permisos administrativos. Durante el proceso, el sistema mostró un error de sintaxis debido a que se ingresó un parámetro incorrecto en el comando. El error ocurrió porque se escribió un valor no válido en lugar de las opciones correctas del comando usermod. Gracias al mensaje mostrado por el sistema, fue posible identificar las banderas adecuadas para añadir usuarios a grupos sin afectar su configuración actual.

Este tipo de errores son comunes en la administración de servidores y es importante interpretar correctamente los mensajes de la terminal para corregir problemas sin comprometer la estabilidad o seguridad del sistema.

Paso 14: Asignación de privilegios elevados y verificación de grupos de seguridad:

Comandos ejecutados:

```
sudo usermod -aG sudo oscar_admin
groups oscar_admin
```

```
admny@debian-ici:~$ sudo usermod -aG sudo oscar_admin
admny@debian-ici:~$ groups oscar_admin
oscar_admin : oscar_admin sudo users
admny@debian-ici:~$
```

Figura 14. Inclusión exitosa del usuario en el grupo sudo.

Paso 15: Actualización de la regla 3-2-1 para respaldos en GNU/Linux

Objetivo de la implementación:

Se actualizó el esquema de respaldos del servidor GNU/Linux para que el proceso ya no dependiera únicamente de la copia local. A partir de esta configuración, el respaldo se genera en Linux, se transfiere automáticamente a Windows Server mediante SCP sobre Tailscale y se sincroniza con Google Drive usando rclone. Con esto se completa la regla 3-2-1 de forma operativa y verificable.

Distribución final de las copias:

- **Copia 1:** /var/backups/local/ en Debian como respaldo local inmediato.
- **Copia 2:** C:\RespaldosLinux en Windows Server como segundo medio mediante SCP/Tailscale.
- **Copia 3:** gdrive:RespaldosLinux como respaldo fuera de sitio mediante rclone.

La implementación mantiene una separación clara entre la copia local, la copia remota y la copia fuera de sitio. Esta distribución permite recuperar información incluso si el servidor principal falla, si se pierde la máquina virtual o si se corrompe el almacenamiento local.

Paso 16: Script de respaldo 3-2-1 en /opt/backup_321.sh: Archivo editado:

```
sudo nano /opt/backup_321.sh
```

El script quedó estructurado con variables de fecha, rutas de almacenamiento y destinos remotos. La primera parte define la ruta local /var/backups/local, el archivo de log/var/log/backup_321.log, el nombre dinámico del respaldo y el destino de Windows Server

```

GNU nano 8.4
#!/bin/bash

FECHA=$(date +%Y-%m-%d_%H-%M-%S)

RUTA_LOCAL="/var/backups/local"
LOG="/var/log/backup_321.log"

# - DB webapi
DB_USER="dbapi"
DB_PASS="webapi2026"
DB_NAME="webapi"
DB_ARCHIVO="respaldo_webapi_${FECHA}.sql.gz"
DB_DESTINO_LOCAL="$RUTA_LOCAL/$DB_ARCHIVO"

WIN_USER="AdminWin"
WIN_HOST="100.118.114.2"
WIN_RUTA="/C:/RespaldosLinux"

RUTA_IMAGENES="/var/local/mis_imagenes"
IMG_ARCHIVO="respaldo_imagenes_${FECHA}.tar.gz"
IMG_DESTINO_LOCAL="$RUTA_LOCAL/$IMG_ARCHIVO"

GDRIVE_DESTINO="gdrive:RespaldosLinux"

echo "===== >> \"$LOG\"
echo "Inicio de respaldo: $(date)" >> "$LOG"

mkdir -p "$RUTA_LOCAL"

# [1/3] Dump local de BD webapi
echo "[1/3] Generando dump de BD webapi..." >> "$LOG"

mysqldump -u "$DB_USER" -p"$DB_PASS" "$DB_NAME" | gzip > "$DB_DESTINO_LOCAL"

if [ $? -ne 0 ]; then
    echo "ERROR: Falló el dump de la BD webapi." >> "$LOG"
    exit 1
fi

tar -czf "$IMG_DESTINO_LOCAL" "$RUTA_IMAGENES" 2>> "$LOG"

if [ $? -ne 0 ]; then
    echo "ERROR: Falló el respaldo de imágenes." >> "$LOG"
    exit 1
fi

echo "Respaldo de imágenes creado: $IMG_DESTINO_LOCAL" >> "$LOG"

# [2/3] Enviando a Windows Server
echo "[2/3] Enviando a Windows Server..." >> "$LOG"

scp "$DB_DESTINO_LOCAL" "$WIN_USER@$WIN_HOST:$WIN_RUTA/"
scp "$IMG_DESTINO_LOCAL" "$WIN_USER@$WIN_HOST:$WIN_RUTA/"

if [ $? -ne 0 ]; then
    echo "ERROR: Falló el envío a Windows Server." >> "$LOG"
    exit 1
fi

echo "Enviado a Windows correctamente." >> "$LOG"

# [3/3] Enviando a Google Drive
echo "[3/3] Enviando a Google Drive..." >> "$LOG"

rclone copy "$DB_DESTINO_LOCAL" "$GDRIVE_DESTINO" >> "$LOG" 2>&1
rclone copy "$IMG_DESTINO_LOCAL" "$GDRIVE_DESTINO" >> "$LOG" 2>&1

if [ $? -ne 0 ]; then
    echo "ERROR: Falló el envío a Google Drive." >> "$LOG"
    exit 1
fi

echo "Enviado a Google Drive correctamente." >> "$LOG"
echo "Respaldo completado: $(date)" >> "$LOG"

```

Figura 15. Configuración inicial del script con variables, ruta local, destino Windows Server y carpeta de Google Drive.

En la segunda parte del script se agregó la transferencia hacia Windows Server mediante scp y la sincronización hacia Google Drive mediante rclone. Para evitar el error de configuración al ejecutar el script con sudo, se indicó explícitamente el archivo de configuración de rclone del usuario admnyc mediante --config /home/admnyc/.config/rclone/rclone.conf.

```

echo "[2/3] Enviando respaldo a Windows Server..." >> "$LOG"
scp "$DESTINO_LOCAL" "$WIN_USER@$WIN_HOST:$WIN_RUTA/" >> "$LOG" 2>&1
if [ $? -ne 0 ]; then
    echo "ERROR: Falló la transferencia hacia Windows Server." >> "$LOG"
    exit 1
fi
echo "Respaldo enviado correctamente a Windows Server." >> "$LOG"
echo "[3/3] Subiendo respaldo a Google Drive..." >> "$LOG"
rclone --config /home/admnyc/.config/rclone/rclone.conf copy "$DESTINO_LOCAL" "$GDRIVE_DESTINO" >> "$LOG" 2>&1
if [ $? -ne 0 ]; then
    echo "ERROR: Falló la subida a Google Drive." >> "$LOG"
    exit 1
fi
echo "Respaldo subido correctamente a Google Drive." >> "$LOG"
echo "Respaldo 3-2-1 finalizado correctamente: $(date)" >> "$LOG"
echo "===== " >> "$LOG"

```

Figura 16. Sección del script encargada de enviar el respaldo a Windows Server y subirlo a Google Drive con rclone.

Paso 17: Autenticación SSH sin contraseña para la copia hacia Windows Server

Comandos ejecutados en Debian:

```

sudo ssh-keygen -t ed25519 -C "backup_linux_root_to_windows"
sudo cat /root/.ssh/id_ed25519.pub

```

```

admynyc@debian-ici:~$ sudo ssh-keygen -t ed25519 -C "backup_linux_root_to_windows"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519
Your public key has been saved in /root/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:w+D/npIH+Yg6X4ICtkgpA5ydyneYjXXbc93qSEiSQ3A backup_linux_root_to_windows
The key's randomart image is:
+--[ED25519 256]--+
|..+o.E
|..=.=+
|. = +.=+ .
|o = .+o+
|+* ..So
|=o. .+.o
|... ..o* .
|.. ..o+.o.
|.+. ++
+-----[SHA256]-----+

```

Figura 17. Generación de llave ED25519 para que root en Debian pueda autenticarse contra Windows Server sin contraseña.

```

admynyc@debian-ici:~$ sudo ssh AdminWin@100.118.114.2
[sudo] contraseña para admynyc:
Microsoft Windows [Versión 10.0.20348.587]
(c) Microsoft Corporation. Todos los derechos reservados.
C:\Users\AdminWin>exit

```

Figura 17A. Prueba de acceso SSH desde Debian hacia Windows Server sin solicitar contraseña.

administrators_authorized_keys. Al tratarse de un usuario administrador en Windows, OpenSSH valida este archivo ubicado en C:\ProgramData\ssh. Posteriormente se ajustaron los permisos con icacfs para que solo SYSTEM y el grupo Administradores conservaran control total sobre el archivo.

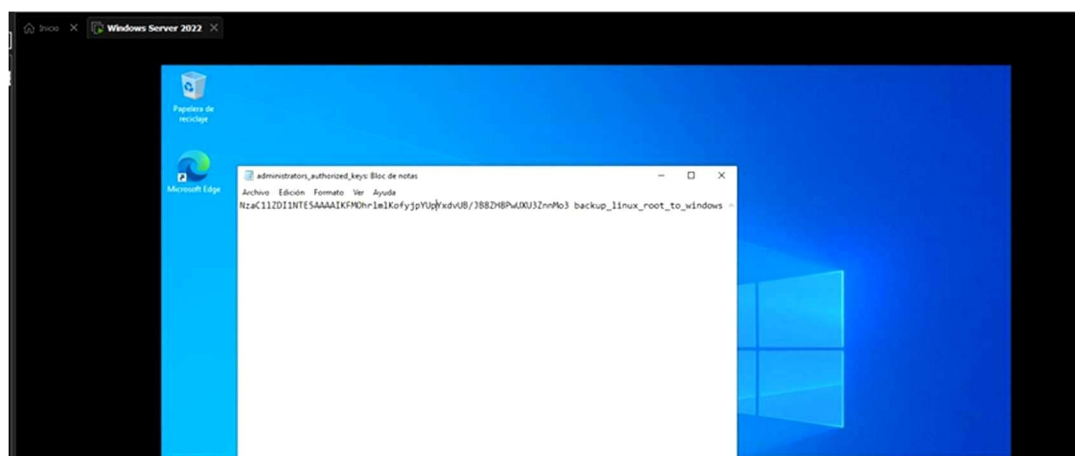


Figura 18. Registro de la llave pública en Windows Server y endurecimiento de permisos del archivo `administrators_authorized_keys`.

Paso 18: Endurecimiento de permisos del script y programación diaria con cron

Comandos ejecutados:

```
sudo chown root:root /opt/backup_321.sh sudo chmod 700 /opt/backup_321.sh
sudo crontab -e
```

El script fue protegido para que solo root pueda modificarlo o ejecutarlo directamente. Esta decisión evita alteraciones no autorizadas en el flujo de respaldo, ya que el archivo contiene rutas, destinos remotos y comandos de transferencia.

Paso 19: Programación diaria del respaldo con cron

La tarea programada se configuró en el crontab de root con la siguiente línea:

```
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
```

La expresión `1 1 * * *` indica que el respaldo se ejecutará todos los días a la 1:01 AM. Se eligió este horario porque normalmente representa una ventana de baja actividad, reduciendo el impacto sobre el servidor y dejando registro de la ejecución en `/var/log/backup_cron.log`.


```

admnyc@debian-ici:~$ sudo crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
admnyc@debian-ici:~$

```

Figura 19. Programación del respaldo automático diario a la 1:01 AM mediante crontab de root.

Paso 20: Prueba manual y validación de la regla 3-2-1

Comando de prueba:

```

sudo /opt/backup_321.sh
sudo tail -n 60 /var/log/backup_321.log

```

La prueba manual confirmó que el respaldo se generó correctamente en Linux, se transfirió a Windows Server y finalmente se subió a Google Drive. En el primer intento, rclone falló porque sudo buscaba la configuración en /root/.config/rclone/rclone.conf; el problema se corrigió indicando el archivo de configuración real del usuario admnyc dentro del script.

Validaciones finales realizadas:

Copia local: ls -lh /var/backups/local/

Copia en Windows Server: ssh AdminWin@100.118.114.2 "dir C:\RespalDOSLinux"

Copia en Google Drive: sudo rclone --config /home/admnyc/.config/rclone/rclone.conf ls gdrive:RespalDOSLinux

```
admny@debian-ici:~$ sudo /opt/backup_321.sh
admny@debian-ici:~$ sudo tail -n 60 /var/log/backup_321.log
=====
Inicio de respaldo 3-2-1: vie 29 may 2026 14:52:11 EDT
[1/3] Generando respaldo local en Linux...
tar: Eliminando la '/' inicial de los nombres
tar: Eliminando la '/' inicial de los objetivos de los enlaces
Respaldo local creado correctamente: /var/backups/local/respaldo_linux_2026-05-29_14-52-11.tar.gz
[2/3] Enviando respaldo a Windows Server...
Warning: Permanently added '100.118.114.2' (ED25519) to the list of known hosts.
Respaldo enviado correctamente a Windows Server.
[3/3] Subiendo respaldo a Google Drive...
2026/05/29 14:54:34 NOTICE: Config file "/root/.config/rclone/rclone.conf" not found - using defaults
2026/05/29 14:54:34 Failed to create file system for "gdrive:RespaldosLinux": didn't find section in config file
ERROR: Falló la subida a Google Drive.
=====
Inicio de respaldo 3-2-1: vie 29 may 2026 15:34:28 EDT
[1/3] Generando respaldo local en Linux...
tar: Eliminando la '/' inicial de los nombres
tar: Eliminando la '/' inicial de los objetivos de los enlaces
Respaldo local creado correctamente: /var/backups/local/respaldo_linux_2026-05-29_15-34-28.tar.gz
[2/3] Enviando respaldo a Windows Server...
Respaldo enviado correctamente a Windows Server.
[3/3] Subiendo respaldo a Google Drive...
Respaldo subido correctamente a Google Drive.
Respaldo 3-2-1 finalizado correctamente: vie 29 may 2026 15:35:50 EDT
=====
admny@debian-ici:~$

admny@debian-ici:~$ ls -lh /var/backups/local/
total 153M
-rw-r--r-- 1 root root 518K may 19 19:36 respaldo_db_2026-05-19_19-36.sql.gz
-rw-r--r-- 1 root root 518K may 20 01:01 respaldo_db_2026-05-20_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 21 01:01 respaldo_db_2026-05-21_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 22 01:01 respaldo_db_2026-05-22_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 23 01:01 respaldo_db_2026-05-23_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 24 01:01 respaldo_db_2026-05-24_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 25 01:01 respaldo_db_2026-05-25_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 25 02:46 respaldo_db_2026-05-25_02-46.sql.gz
-rw-r--r-- 1 root root 518K may 26 01:01 respaldo_db_2026-05-26_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 27 01:01 respaldo_db_2026-05-27_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 28 01:01 respaldo_db_2026-05-28_01-01.sql.gz
-rw-r--r-- 1 root root 518K may 29 01:01 respaldo_db_2026-05-29_01-01.sql.gz
-rw-r--r-- 1 root root 74M may 29 14:52 respaldo_linux_2026-05-29_14-52-11.tar.gz
-rw-r--r-- 1 root root 74M may 29 15:34 respaldo_linux_2026-05-29_15-34-28.tar.gz
admny@debian-ici:~$ ssh AdminWin@100.118.114.2 "dir C:\RespaldosLinux"
AdminWin@100.118.114.2's password:
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 9298-961C

Directorio de C:\RespaldosLinux

29/05/2026 02:34 p. m. <DIR> .
29/05/2026 01:43 p. m. 43 prueba_linux.txt
29/05/2026 01:54 p. m. 76,824,687 respaldo_linux_2026-05-29_14-52-11.tar.gz
29/05/2026 02:35 p. m. 76,824,637 respaldo_linux_2026-05-29_15-34-28.tar.gz
3 archivos 153,649,367 bytes
1 dirs 43,414,376,448 bytes libres
admny@debian-ici:~$ sudo rclone --config /home/admny/.config/rclone/rclone.conf ls gdrive:RespaldosLinux
76824637 respaldo_linux_2026-05-29_15-34-28.tar.gz
admny@debian-ici:~$
```

Figura 20. Validación final del respaldo 3-2-1: ejecución correcta, copia local, copia en Windows Server y copia fuera de sitio en Google Drive.

Con esta configuración, el servidor GNU/Linux queda respaldado diariamente bajo el esquema 3-2-1: una copia local para restauración inmediata, una copia remota en Windows Server como segundo medio y una copia externa en Google Drive para recuperación ante pérdida del entorno principal.

Paso 22: Instalación de Google Authenticator para autenticación de dos factores (TOTP):

Comandos ejecutados:

```
sudo apt update && sudo apt install libpam-google-authenticator -y && google-authenticator
```

Se instaló el paquete libpam-google-authenticator en el servidor Debian mediante apt y se inició el asistente de configuración con el comando google-authenticator. Este módulo permite integrar la autenticación de doble factor basada en contraseñas de un solo uso con temporización (TOTP) al servicio SSH. La instalación de este paquete es el primer paso para añadir una capa adicional de

seguridad que requiere tanto la llave criptográfica como un código temporal generado por una aplicación de autenticación.

Paso 23: Generación del código QR y clave secreta TOTP:



Código QR y clave TOTP ocultos por seguridad



Figura 31. Código QR y clave secreta TOTP generados para vincular la aplicación autenticadora.

El asistente generó un código QR y una clave secreta única para vincular la cuenta del usuario con una aplicación de autenticación como Google Authenticator o Authy. Por seguridad, la clave y el código QR se mantienen resguardados fuera del reporte. Al escanear el código desde la aplicación móvil, esta queda enlazada al servidor y comienza a generar códigos TOTP cada 30 segundos.

Paso 24: Configuración de opciones de seguridad TOTP:


```

Do you want me to update your "/home/admnyc/.google_authenticator" file? (y/n) y

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y

By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) y

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y
admnc@debian-ici:~$

```

Figura 32. Configuración de políticas de seguridad del módulo TOTP.

El asistente presentó varias preguntas de configuración para ajustar el comportamiento del módulo TOTP. Se respondió afirmativamente a todas para habilitar: tokens basados en tiempo, escritura del archivo de configuración, restricción de un solo uso por token (evitando ataques de repetición), ventana de tiempo ampliada para compensar diferencias de reloj entre cliente y servidor, y limitación de intentos de autenticación a máximo 3 cada 30 segundos para mitigar ataques de fuerza bruta.

Paso 25: Confirmación del código y obtención de códigos de emergencia:

```

code entered (correct code goes in) again
Enter code from app (-1 to skip): 121106
Code confirmed
[Redacted area]
Do you want me to update your "/home/admnyc/.google_authenticator" file? (y/n)

```

Figura 33. Confirmación del código TOTP y códigos de emergencia de un solo uso.

Se confirmó el correcto funcionamiento del módulo ingresando un código generado por la aplicación autenticadora. Al validarse exitosamente, el sistema proporcionó cinco códigos de emergencia de un solo uso (scratch codes) que permiten acceder al servidor en caso de que el dispositivo móvil no esté disponible. Estos códigos deben guardarse en un lugar seguro fuera del servidor, ya que son de uso único y no pueden regenerarse sin reconfigurar el módulo.

Paso 26: Revisión del archivo de configuración PAM para SSH (antes de modificar):

Archivo modificado:

```
/etc/pam.d/ssh
```

A screenshot of a terminal window showing the original content of the /etc/pam.d/sshd file. The file is being viewed in the nano editor. The content includes standard PAM configuration for SSH, such as including common-auth, common-account, and common-session modules, and setting session rules for pam_selinux, pam_loginuid, and pam_keyinit. The terminal window has a dark background with a Kali Linux logo watermark. The nano editor's interface is visible at the top and bottom of the terminal window.

Figura 34. Contenido original del archivo de configuración PAM para SSH.

Se abrió el archivo /etc/pam.d/sshd con el editor Nano para revisar su contenido antes de realizar modificaciones. Este archivo controla los módulos de autenticación conectables (PAM) que utiliza el servicio SSH. Conocer su estructura original es fundamental para agregar correctamente la línea del módulo Google Authenticator sin romper el flujo de autenticación existente.

Paso 27: Integración del módulo Google Authenticator en PAM para SSH:

Archivo modificado:

/etc/pam.d/sshd (Primera línea agregada: auth required pam_google_authenticator.so nullok)

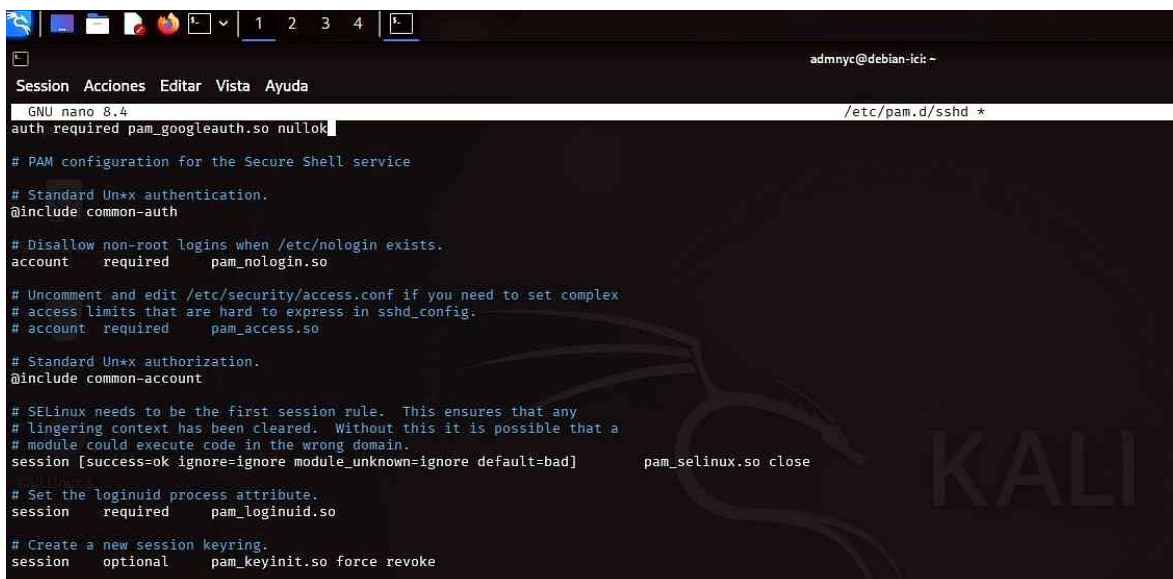
A screenshot of the terminal window showing the modified /etc/pam.d/sshd file. The first line of the file has been changed to 'auth required pam_google_authenticator.so nullok'. The rest of the file content remains the same as in the original image. The nano editor's interface is visible at the top and bottom of the terminal window.

Figura 35. Adición de la directiva `pam_google_authenticator` al inicio del archivo PAM SSH.

Se agregó la línea `auth required pam_google_authenticator.so nullok` al inicio del archivo `/etc/pam.d/sshd`. Esta directiva indica a PAM que el módulo de Google Authenticator es obligatorio durante el proceso de autenticación SSH. La opción `nullok` permite que usuarios que aún no hayan configurado el autenticador puedan seguir iniciando sesión, facilitando una transición gradual. Una vez que todos los usuarios estén configurados, esta opción debe eliminarse para hacer el segundo factor completamente obligatorio.

Paso 28: Habilitación de autenticación por teclado interactivo en `sshd_config` para TOTP:

Archivo modificado:

`/etc/ssh/sshd_config` (Líneas: `KbdInteractiveAuthentication yes` y `AuthenticationMethods publickey,keyboard-interactive`)

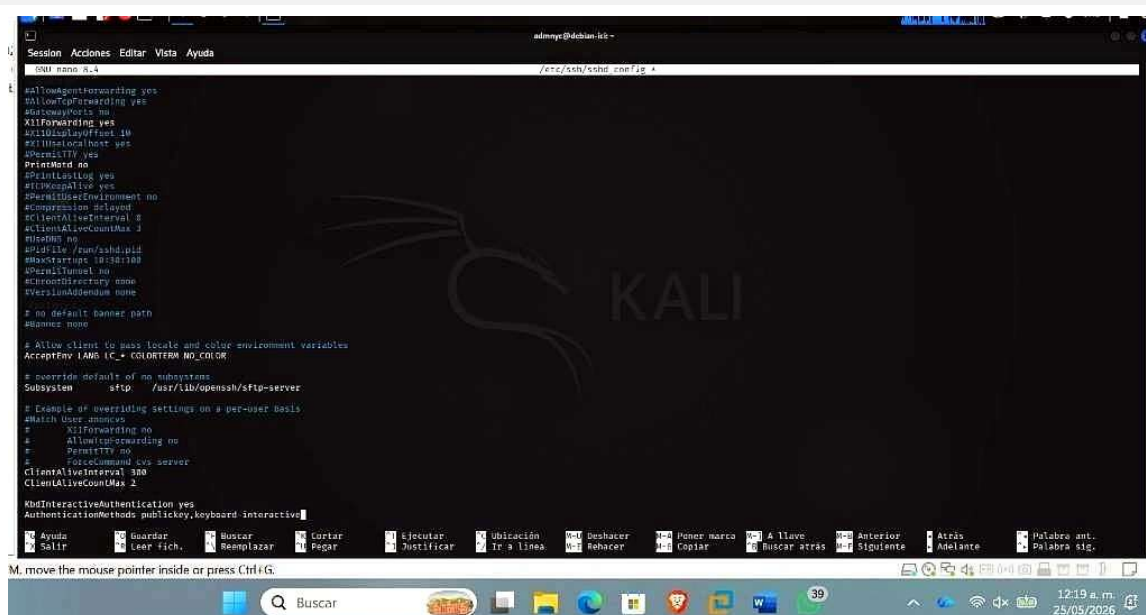


Figura 36. Configuración de métodos de autenticación combinados en `sshd_config` para habilitar TOTP.

Se modificó el archivo de configuración principal de SSH para habilitar la autenticación por teclado interactivo (`KbdInteractiveAuthentication yes`), necesaria para que el servidor pueda solicitar el código TOTP durante el proceso de inicio de sesión. Adicionalmente, se configuró `AuthenticationMethods publickey,keyboard-interactive` para exigir que el usuario supere ambos factores en secuencia: primero la llave criptográfica y luego el código TOTP generado por la aplicación autenticadora.

Paso 29: Validación de la configuración y corrección del método de autenticación:

Comandos ejecutados:

```
sudo sshd -t
sudo systemctl restart ssh
```



```
admnyca@debian-ici:~$ sudo sshd -t && sudo systemctl restart ssh
Disabled method "keyboard-interactive" in AuthenticationMethods list "publickey,keyboard-interactive"
AuthenticationMethods cannot be satisfied by enabled authentication methods
admnyca@debian-ici:~$
```

Figura 37. Advertencia del validador sshd sobre conflicto en los métodos de autenticación configurados.

Al validar la configuración con `sudo sshd -t`, el sistema reportó una advertencia indicando que el método `keyboard-interactive` había sido deshabilitado porque no estaba satisfecho por los métodos de autenticación habilitados. Esto se debió a un conflicto entre la directiva `AuthenticationMethods publickey,keyboard-interactive` y la configuración de `PasswordAuthentication no`. Para corregirlo, se procedió a ajustar la configuración de SSH a fin de permitir que el flujo interactivo de TOTP funcione correctamente sin depender de contraseñas tradicionales.

Paso 30: Corrección final habilitando `ChallengeResponseAuthentication` para TOTP:

Archivo modificado:

`/etc/ssh/sshd_config` (Líneas: `PasswordAuthentication no`, `KbdInteractiveAuthentication yes` y `ChallengeResponseAuthentication yes`)

```
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to "no" here!
PasswordAuthentication no
PermitEmptyPasswords no

# Change to "yes" to enable keyboard-interactive authentication. Depending on
# the system's configuration, this may involve passwords, challenge-response,
# one-time passwords or some combination of these and other methods.
# Beware issues with some PAM modules and threads.
KbdInteractiveAuthentication yes
ChallengeResponseAuthentication yes

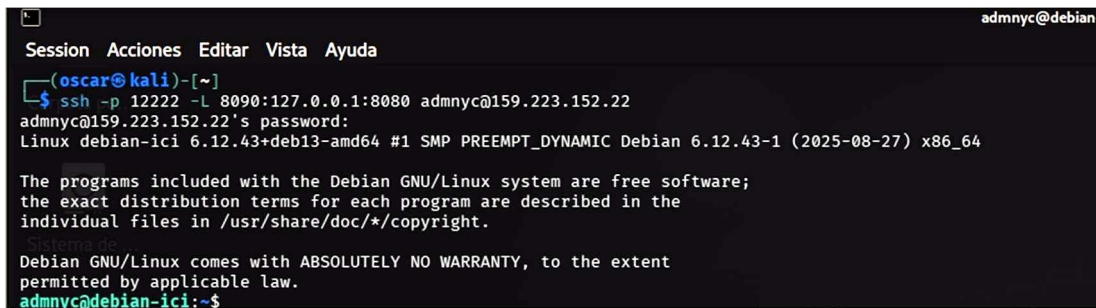
# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes
```

Figura 38. Configuración final de autenticación en `sshd_config` con TOTP habilitado correctamente.

Se corrigió la configuración de SSH habilitando `ChallengeResponseAuthentication yes`, lo cual permite que el mecanismo PAM pueda solicitar el código TOTP de forma interactiva durante el inicio de sesión. Se mantuvo `PasswordAuthentication no` para seguir bloqueando el acceso mediante contraseñas tradicionales, garantizando que el único flujo válido sea: llave criptográfica + código TOTP. Con esta configuración, el servidor implementa autenticación de doble factor (2FA) completa, elevando significativamente la seguridad del acceso remoto al exigir algo que el usuario tiene (llave SSH) y algo que el usuario posee en tiempo real (código TOTP de la aplicación autenticadora).

Comandos de revisión (Todo adentro del servidor):

Desde tu Kali Linux (Abrir el túnel y conectarte): `ssh -p 12222 -L 8090:127.0.0.1:8080 admnyc@159.223.152.22`



```
Session Acciones Editar Vista Ayuda
(oscar@kali)-[~]
$ ssh -p 12222 -L 8090:127.0.0.1:8080 admnyc@159.223.152.22
admynyc@159.223.152.22's password:
Linux debian-ici 6.12.43+deb13-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.43-1 (2025-08-27) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
admynyc@debian-ici:~$
```

Figura 22. Apertura de túnel local y conexión remota SSH.

Justificación del uso de Túnel SSH En esta práctica se implementó un túnel SSH como medida de seguridad para proteger los servicios internos del servidor. En lugar de dejar puertos como el 8080 expuestos a internet, se mantuvieron bloqueados mediante el firewall, permitiendo el acceso únicamente a través de una conexión SSH autenticada.

El firewall (UFW) mantiene una política restrictiva bloqueando el acceso externo a dichos puertos, reduciendo así la superficie de ataque del servidor. Además, todo el flujo de datos entre el cliente (Kali Linux) y el servidor viaja cifrado mediante el protocolo SSH, evitando ataques de interceptación de datos tipo Man-in-the-Middle.

Finalmente, este método garantiza que solo los administradores autorizados que hayan superado la autenticación por llaves criptográficas puedan interactuar con los servicios internos del servidor.

1. Revisar el Firewall (UFW): `sudo ufw status verbose`



```
admynyc@debian-ici:~$ sudo ufw status verbose
[sudo] contraseña para admynyc:
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
12222 ALLOW IN Anywhere
8080 ALLOW IN Anywhere
22/tcp ALLOW IN Anywhere
12222/tcp ALLOW IN Anywhere # Acceso SSH Administrativo
12222 (v6) ALLOW IN Anywhere (v6)
8080 (v6) ALLOW IN Anywhere (v6)
22/tcp (v6) ALLOW IN Anywhere (v6)
12222/tcp (v6) ALLOW IN Anywhere (v6) # Acceso SSH Administrativo

admynyc@debian-ici:~$
```

Figura 23. Auditoría del estado y reglas activas en UFW.

El comando `sudo ufw status verbose` muestra de forma detallada el estado del firewall, incluyendo políticas globales y reglas activas. Permite verificar qué puertos y protocolos están abiertos, como 12222/tcp para acceso SSH administrativo.

2. Revisar la Cuenta de Usuario Nueva:

```
groups oscar_admin
```

```
admny@debian-ici:~$ groups oscar_admin
oscar_admin : oscar_admin sudo users
```

Figura 24. Verificación de grupos asociados al nuevo usuario.

El comando `groups oscar_admin` permite verificar los grupos y privilegios asignados a la cuenta en el sistema. La salida confirma que el usuario pertenece al grupo `sudo`, habilitando la ejecución de tareas administrativas de forma controlada.

3. Revisar el Respaldo Automático (Cron):

```
sudo crontab -l
```

```
admny@debian-ici:~$ sudo crontab -l
[sudo] contraseña para admny:
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
1 1 * * * /bin/bash /opt/backup_321.sh >> /var/log/backup_cron.log 2>&1
admny@debian-ici:~$
```

Figura 25. Listado y verificación de tareas cron activas.

El comando `sudo crontab -l` permite visualizar las tareas programadas del usuario `root` en el servicio Cron. Se utiliza para verificar que el script de respaldos y sus registros hayan sido configurados correctamente para su ejecución automática.

4. Revisar el Hardening de SSH (El blindaje):

```
sudo grep -E "PermitRootLogin|PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config
```

```
admny@debian-ici:~$ sudo grep -E "PermitRootLogin|PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config
PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
# PasswordAuthentication. Depending on your PAM configuration,
# the setting of "PermitRootLogin yes
# PAM authentication, then enable this but set PasswordAuthentication
admny@debian-ici:~$
```

Figura 27. Auditoría de directivas de endurecimiento en `sshd_config`.

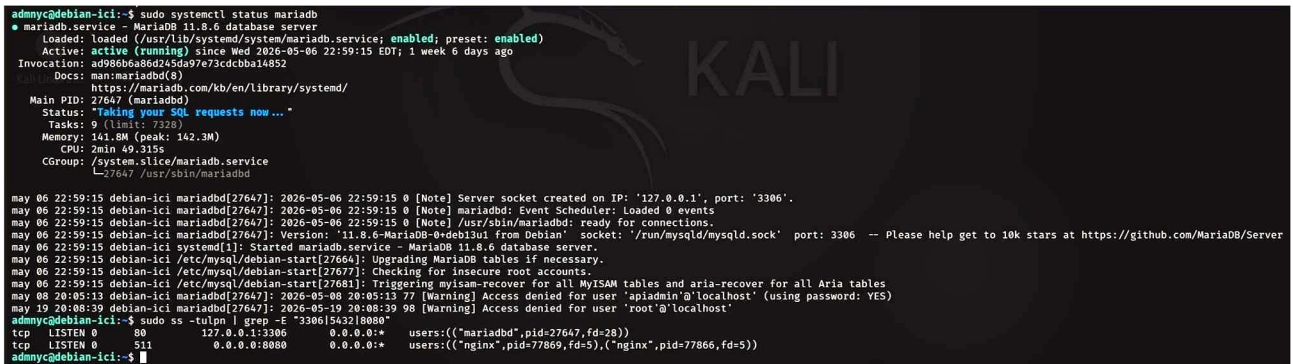
El comando `sudo grep -E "PermitRootLogin| PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config` permite verificar las configuraciones críticas de seguridad de SSH, confirmando que el acceso root y la autenticación por contraseña están deshabilitados y que únicamente se permite el acceso mediante llaves criptográficas.

6. Revisar el estado de las Bases de Datos (SQL):

Ver el estado de MariaDB - `sudo systemctl status mariadb`

Ver qué puertos están escuchando de forma interna y segura - `sudo ss -tulpn | grep`

-E "3306|5432|8080"



```
admin@kali:~$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-06 22:59:15 EDT; 1 week 6 days ago
     Invocation: ad986b6a86d245da97e73cdbcba14852
       Docs: man:mariadb(8)
            https://mariadb.com/kb/en/library/systemd/
   Main PID: 27647 (mariadb)
  Status: "Taking your SQL requests now ..."
    Tasks: 9 (limit: 7328)
   Memory: 141.8M (peak: 142.3M)
      CPU: 2min 49.315s
   CGroup: /system.slice/mariadb.service
           └─27647 /usr/sbin/mariadb

may 06 22:59:15 debian-ic1 mariadb[27647]: 2026-05-06 22:59:15 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
may 06 22:59:15 debian-ic1 mariadb[27647]: 2026-05-06 22:59:15 0 [Note] mariadb: Event Scheduler: Loaded 0 events
may 06 22:59:15 debian-ic1 mariadb[27647]: 2026-05-06 22:59:15 0 [Note] /usr/sbin/mariadb: ready for connections.
may 06 22:59:15 debian-ic1 mariadb[27647]: Version: '11.8.6-MariaDB-0+deb13u1 from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help get to 10k stars at https://github.com/MariaDB/Server
may 06 22:59:15 debian-ic1 systemd[1]: Started mariadb.service - MariaDB 11.8.6 database server.
may 06 22:59:15 debian-ic1 /etc/mysql/debian-start[27664]: Upgrading MariaDB tables if necessary.
may 06 22:59:15 debian-ic1 /etc/mysql/debian-start[27677]: Checking for insecure root accounts.
may 06 22:59:15 debian-ic1 /etc/mysql/debian-start[27681]: Triggering myisam-recover for all MyISAM tables and aria-recover for all Aria tables
may 08 20:05:13 debian-ic1 mariadb[27647]: 2026-05-08 20:05:13 77 [Warning] Access denied for user 'apiadmin@localhost' (using password: YES)
may 19 20:08:39 debian-ic1 mariadb[27647]: 2026-05-19 20:08:39 98 [Warning] Access denied for user 'root@localhost'
admin@kali:~$ sudo ss -tulpn | grep -E "3306|5432|8080"
tcp        LISTEN 0      80          127.0.0.1:3306      0.0.0.0:*           users:((("mariadb",pid=27647,fd=28))
tcp        LISTEN 0      511          0.0.0.0:8080        0.0.0.0:*           users:((("nginx",pid=77869,fd=5),("nginx",pid=77866,fd=5))
```

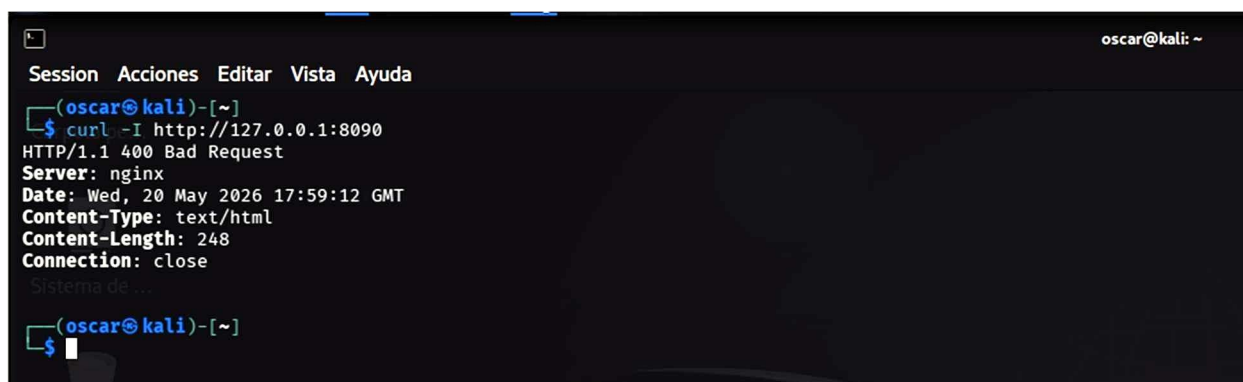
Figura 28. Auditoría del servicio MariaDB y sockets de red activos.

Los comandos `sudo systemctl status mariadb` y `sudo ss -tulpn | grep -E "3306| 5432|8080"` permiten verificar que el servicio MariaDB se encuentra activo y que el puerto 3306 está escuchando únicamente en 127.0.0.1, garantizando su funcionamiento interno sin exposición directa a redes externas.

7. Probar el túnel (Desde tu Kali Linux otra vez):

Para cerrar la prueba demostrando que el túnel que abriste en el Paso 1 funciona:

- Abre otra pestaña o ventana de la terminal en tu Kali Linux (esta no debe estar conectada al servidor, debe decir kali@kali).
- Haz un escaneo rápido al puerto local 8090 de tu Kali para demostrar que estás mapeando el puerto 8080 del servidor de forma segura:



```
ooscar@kali: ~  
Session Acciones Editar Vista Ayuda  
(oscar@kali)-[~]  
$ curl -I http://127.0.0.1:8090  
HTTP/1.1 400 Bad Request  
Server: nginx  
Date: Wed, 20 May 2026 17:59:12 GMT  
Content-Type: text/html  
Content-Length: 248  
Connection: close  
Sistema de ...  
(oscar@kali)-[~]  
$
```

Figura 23. Prueba local de conectividad HTTP mediante comando curl.

Para comprobar el funcionamiento del firewall y del túnel SSH, se realizó una petición HTTP con curl -I desde Kali Linux hacia 127.0.0.1:8090. La respuesta recibida del servidor nginx confirmó que el tráfico viajó correctamente a través del túnel SSH cifrado.

Aunque se obtuvo un código HTTP 400 (Bad Request), esto indica que la conexión con el servicio interno fue exitosa y que el puerto remoto 8080 es accesible únicamente mediante el túnel seguro, permaneciendo bloqueado para el acceso público gracias a las reglas del firewall.

Con esta prueba se valida que la arquitectura implementada reduce la superficie de ataque y permite administrar el servicio de forma segura sin exponer puertos innecesarios a internet.

¿Un usuario puede entrar con una contraseña y si no por qué?

No, el usuario no puede entrar con contraseña porque el servicio SSH fue configurado para permitir únicamente autenticación mediante llaves criptográficas públicas y privadas. Esto se logró deshabilitando la opción PasswordAuthentication no en el archivo de configuración de SSH.

Gracias a esta medida de hardening, el servidor rechaza cualquier intento de acceso con contraseña, aumentando la seguridad al evitar ataques de fuerza bruta, robo de credenciales y uso de contraseñas débiles.

Conclusión

La práctica de hardening de GNU/Linux permitió fortalecer el servidor desde diferentes capas de seguridad: acceso remoto, autenticación, privilegios, firewall, respaldo y verificación operativa. Se protegió el servicio SSH mediante cambio de puerto, bloqueo de acceso directo como root, autenticación por llaves criptográficas y tiempos de desconexión por inactividad. Además, se configuró UFW bajo una política restrictiva, se creó un usuario nominal con privilegios controlados y se documentó el uso de autenticación multifactor mediante Google Authenticator para elevar la seguridad del acceso remoto. También se completó la regla 3-2-1 de respaldos, generando una copia local en Debian, una copia remota en Windows Server por medio de SCP sobre Tailscale y una copia externa en Google Drive mediante rclone. La programación con cron a la 1:01 AM permite que el proceso se ejecute de forma automática y auditable, reduciendo la dependencia de acciones manuales. En conjunto, las medidas implementadas demuestran una mejora integral en la protección, disponibilidad y recuperación del servidor GNU/Linux, manteniendo evidencia clara de cada configuración aplicada.